

PRACTICAL WEEK 4

PART - 1 : Create Database

DROP DATABASE IF EXISTS bookflow_db;

CREATE DATABASE bookflow_db;

USE bookflow_db;

OUTPUT:

The screenshot displays a database management tool interface. On the left, a 'SCHEMAS' panel shows a tree view with databases: 2520080069_ananya, bankdb, kludb, and sys. Below this, tabs for 'Administration', 'Schemas', and 'Information' are visible, with 'Schemas' selected. The main area shows a list of SQL statements executed:

- 1 • DROP DATABASE IF EXISTS bookflow_db;
- 2 • CREATE DATABASE bookflow_db;
- 3 •
- 4 • USE bookflow_db;

Below the statements, an 'Output' panel shows the results of the execution:

#	Time	Action	Message	Duration / Fetch
✓ 1	14:26:17	DROP DATABASE IF EXISTS bookflow_db	4 row(s) affected	0.250 sec
⚠ 2	14:26:31	DROP DATABASE IF EXISTS bookflow_db	0 row(s) affected, 1 warning(s): 1008 Can't drop databa...	0.016 sec
✓ 3	14:26:31	CREATE DATABASE bookflow_db	1 row(s) affected	0.015 sec
✓ 4	14:26:48	USE bookflow_db	0 row(s) affected	0.000 sec

On the right side of the interface, a context help message is displayed: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

PART - 2 : Create books table

CREATE TABLE Books

```
(  
    book_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    author VARCHAR(150) NOT NULL,  
    genre VARCHAR(100),  
    description TEXT,  
    published_year INT  
);
```

OUTPUT:

The screenshot displays a database management interface with a left sidebar showing a schema tree. The main editor area contains the following SQL script:

```
1 DROP DATABASE IF EXISTS bookflow_db;  
2 CREATE DATABASE bookflow_db;  
3  
4 USE bookflow_db;  
5  
6  
7 CREATE TABLE Books  
8 (  
9     book_id INT AUTO_INCREMENT PRIMARY KEY,  
10  
11     title VARCHAR(200) NOT NULL,  
12  
13     author VARCHAR(150) NOT NULL,  
14  
15     genre VARCHAR(100),  
16  
17     description TEXT,  
18  
19     published_year INT  
20 );  
21
```

On the right side, a context help message is displayed: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

At the bottom, the "Output" pane shows the "Action Output" table:

#	Time	Action	Message	Duration / Fetch
✓ 1	14:26:17	DROP DATABASE IF EXISTS bookflow_db	4 row(s) affected	0.250 sec
⚠ 2	14:26:31	DROP DATABASE IF EXISTS bookflow_db	0 row(s) affected, 1 warning(s): 1008 Can't drop databa...	0.016 sec
✓ 3	14:26:31	CREATE DATABASE bookflow_db	1 row(s) affected	0.015 sec
✓ 4	14:26:48	USE bookflow_db	0 row(s) affected	0.000 sec
✓ 5	14:32:26	CREATE TABLE Books (book_id INT AUTO_INCR...	0 row(s) affected	0.046 sec

PART- 3: Insert book data

INSERT INTO Books

```
(
    title,
    author,
    genre,
    description,
    published_year
)
VALUES
(
    'The Martian',
    'Andy Weir',
    'Science Fiction',
    'An astronaut struggles to survive alone on Mars.',
    2011
),
(
    'Dune',
    'Frank Herbert',
    'Science Fiction',
    'A story involving planets, politics, survival and space.',
    1965
),
(
    'The Hitchhikers Guide to the Galaxy',
    'Douglas Adams',
    'Science Fiction',
    'A humorous adventure across space and alien worlds.',
    1979
),
(
    'Pride and Prejudice',
    'Jane Austen',
    'Romance',
    'A classic story about relationships and social expectations.',
    1813
),
(
    'The Great Gatsby',
```

'F. Scott Fitzgerald',
'Classic',
'A story of ambition, wealth, love and the American dream.',
1925

),

(

'Foundation',
'Isaac Asimov',
'Science Fiction',
'A futuristic story about mathematics, civilization and space.',
1951

),

(

'The Hobbit',
'J.R.R. Tolkien',
'Fantasy',
'A fantasy adventure involving Bilbo Baggins and a dangerous journey.',
1937

),

(

'Enders Game',
'Orson Scott Card',
'Science Fiction',
'A young student is trained for an interstellar conflict.',
1985

),

(

'Harry Potter and the Sorcerers Stone',
'J.K. Rowling',
'Fantasy',
'A young wizard begins his magical education.',
1997

),

(

'Twenty Thousand Leagues Under the Seas',
'Jules Verne',
'Adventure',
'A fantastic underwater adventure aboard a mysterious submarine.',
1870

);

OUTPUT:

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' panel lists databases: 2520080069_ananya, bankdb, kludb, and sys. Below it, the 'Administration' and 'Schemas' tabs are visible, with 'No object selected' displayed. The main editor shows a SQL script with three INSERT statements for a 'Books' table. The script is as follows:

```
85      1937
86      ),
87
88      (
89          'Enders Game',
90          'Orson Scott Card',
91          'Science Fiction',
92          'A young student is trained for an interstellar conflict.',
93          1985
94      ),
95
96      (
97          'Harry Potter and the Sorcerers Stone',
98          'J.K. Rowling',
99          'Fantasy',
100         'A young wizard begins his magical education.',
101         1997
102     ),
103
104     (
105         'Twenty Thousand Leagues Under the Seas',
106         'Jules Verne',
107         'Adventure',
108         'A fantastic underwater adventure aboard a mysterious submarine',
109         1870
110     );
111
```

On the right, a help message states: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

At the bottom, the 'Output' window shows the 'Action Output' table:

#	Time	Action	Message	Duration / Fetch
✓ 1	14:26:17	DROP DATABASE IF EXISTS bookflow_db	4 row(s) affected	0.250 sec
⚠ 2	14:26:31	DROP DATABASE IF EXISTS bookflow_db	0 row(s) affected, 1 warning(s): 1008 Can't drop databa...	0.016 sec
✓ 3	14:26:31	CREATE DATABASE bookflow_db	1 row(s) affected	0.015 sec
✓ 4	14:26:48	USE bookflow_db	0 row(s) affected	0.000 sec
✓ 5	14:32:26	CREATE TABLE Books (book_id INT AUTO_INCR...	0 row(s) affected	0.046 sec
✓ 6	14:34:36	INSERT INTO Books (title, author, genre, de...	10 row(s) affected Records: 10 Duplicates: 0 Warning...	0.032 sec

PART-4: Display books

```
SELECT
    book_id,
    title,
    author,
    genre,
    published_year
FROM Books
ORDER BY book_id;
```

OUTPUT:

The screenshot shows a database management interface with a 'Navigator' pane on the left, a 'Query 1' editor in the center, and a 'Result Grid' at the bottom. The 'Navigator' pane shows a list of schemas: 2520080069_ananya, bankdb, kludb, and sys. The 'Query 1' editor contains a SQL query that inserts a new book record and then selects all records from the 'Books' table, ordered by 'book_id'. The 'Result Grid' displays the output of the query, showing 10 rows of book data.

Query 1

```
104 (
105     'Twenty Thousand Leagues Under the Seas',
106     'Jules Verne',
107     'Adventure',
108     'A fantastic underwater adventure aboard a mysterious submarine.',
109     1870
110 );
111
112 • SELECT
113     book_id,
114     title,
115     author,
116     genre,
117     published_year
118 FROM Books
119 ORDER BY book_id;
```

Result Grid

	book_id	title	author	genre	published_year
▶	1	The Martian	Andy Weir	Science Fiction	2011
	2	Dune	Frank Herbert	Science Fiction	1965
	3	The Hitchhikers Guide to the Galaxy	Douglas Adams	Science Fiction	1979
	4	Pride and Prejudice	Jane Austen	Romance	1813
	5	The Great Gatsby	F. Scott Fitzgerald	Classic	1925
	6	Foundation	Isaac Asimov	Science Fiction	1951
	7	The Hobbit	J.R.R. Tolkien	Fantasy	1937
	8	Enders Game	Orson Scott Card	Science Fiction	1985
	9	Harry Potter and the Sorcerers Stone	J.K. Rowling	Fantasy	1997
	10	Twenty Thousand Leagues Under the Seas	Jules Verne	Adventure	1870
★	NULL	NULL	NULL	NULL	NULL

PART - 5: Add embedding column

ALTER TABLE Books

ADD COLUMN embedding JSON;

DESCRIBE Books;

OUTPUT:

The screenshot shows a database management interface with a SQL editor and a result grid. The SQL editor contains the following queries:

```
109      1870
110  );
111
112  • SELECT
113      book_id,
114      title,
115      author,
116      genre,
117      published_year
118  FROM Books
119  ORDER BY book_id;
120
121  • ALTER TABLE Books
122  ADD COLUMN embedding JSON;
123
124  • DESCRIBE Books;
```

The result grid displays the structure of the 'Books' table:

Field	Type	Null	Key	Default	Extra
book_id	int	NO	PRI	NULL	auto_increment
title	varchar(200)	NO		NULL	
author	varchar(150)	NO		NULL	
genre	varchar(100)	YES		NULL	
description	text	YES		NULL	
published_year	int	YES		NULL	
embedding	json	YES		NULL	

The interface also shows a 'Navigator' panel on the left with a 'SCHEMAS' section containing a list of databases: 2520080069_ananya, bankdb, kludb, and sys. The 'Administration' and 'Schemas' tabs are visible at the bottom. The 'Result Grid' tab is selected, and the 'Filter Rows' field is empty. The 'Export' button is visible, and the 'Wrap Cell Content' checkbox is checked. The 'Result Grid' button is highlighted in blue. The 'Form Editor' and 'Field Types' buttons are also visible. The 'Result 2' tab is active at the bottom, and the 'Read Only' status is indicated.

PART - 6: Insert mock vectors

```
UPDATE Books
SET embedding = JSON_ARRAY(0.90, 0.80, 0.20)
WHERE book_id = 1;
UPDATE Books
SET embedding = JSON_ARRAY(0.85, 0.75, 0.25)
WHERE book_id = 2;
UPDATE Books
SET embedding = JSON_ARRAY(0.80, 0.70, 0.30)
WHERE book_id = 3;
UPDATE Books
SET embedding = JSON_ARRAY(0.10, 0.20, 0.90)
WHERE book_id = 4;
UPDATE Books
SET embedding = JSON_ARRAY(0.15, 0.30, 0.80)
WHERE book_id = 5;
UPDATE Books
SET embedding = JSON_ARRAY(0.95, 0.65, 0.20)
WHERE book_id = 6;
UPDATE Books
SET embedding = JSON_ARRAY(0.30, 0.75, 0.90)
WHERE book_id = 7;
UPDATE Books
SET embedding = JSON_ARRAY(0.88, 0.85, 0.15)
WHERE book_id = 8;
UPDATE Books
SET embedding = JSON_ARRAY(0.20, 0.55, 0.95)
WHERE book_id = 9;
UPDATE Books
SET embedding = JSON_ARRAY(0.70, 0.90, 0.40)
WHERE book_id = 10;
```


OUTPUT:

Output					
Context Help Snippets					
Output					
Action Output					
	#	Time	Action	Message	Duration / Fetch
✓	12	14:39:24	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.000 sec
✓	13	14:39:36	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.000 sec
✓	14	14:39:44	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.000 sec
✓	15	14:39:56	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.015 sec
✓	16	14:40:03	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.016 sec
✓	17	14:40:12	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.015 sec
✓	18	14:40:22	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.000 sec
✓	19	14:40:35	UPDATE Books SET embedding = JSON_ARRAY(0...	1 row(s) affected Rows matched: 1 Changed: 1 Wa...	0.000 sec

PART - 7 : Verify all the vectors

```
SELECT
    book_id,
    title,
    embedding
FROM Books
ORDER BY book_id;
OUTPUT:
```

The screenshot shows a database management interface with a 'Navigator' pane on the left, a 'Query 1' editor in the center, and a 'Result Grid' at the bottom. The 'Navigator' pane shows a tree of schemas: 2520080069_ananya, bankdb, kludb, and sys. The 'Query 1' editor contains SQL code for updating book embeddings and a SELECT query to verify them. The 'Result Grid' displays the output of the SELECT query, showing columns for book_id, title, and embedding.

Query 1: Skill3_Views_triggers_procedur...

```
147 • UPDATE Books
148     SET embedding = JSON_ARRAY(0.88, 0.85, 0.15)
149     WHERE book_id = 8;
150 • UPDATE Books
151     SET embedding = JSON_ARRAY(0.20, 0.55, 0.95)
152     WHERE book_id = 9;
153 • UPDATE Books
154     SET embedding = JSON_ARRAY(0.70, 0.90, 0.40)
155     WHERE book_id = 10;
156
157 • SELECT
158     book_id,
159     title,
160     embedding
161 FROM Books
162 ORDER BY book_id;
```

Result Grid

	book_id	title	embedding
▶	1	The Martian	[0.90, 0.80, 0.20]
	2	Dune	[0.85, 0.75, 0.25]
	3	The Hitchhikers Guide to the Galaxy	[0.80, 0.70, 0.30]
	4	Pride and Prejudice	[0.10, 0.20, 0.90]
	5	The Great Gatsby	[0.15, 0.30, 0.80]
	6	Foundation	[0.95, 0.65, 0.20]
	7	The Hobbit	[0.30, 0.75, 0.90]
	8	Enders Game	[0.88, 0.85, 0.15]
	9	Harry Potter and the Sorcerers Stone	[0.20, 0.55, 0.95]
	10	Twenty Thousand Leagues Under the Seas	[0.70, 0.90, 0.40]
*	NULL	NULL	NULL

PART-8 : Search vector

The instructor provides a search vector.

For this practical we use:

[0.90, 0.80, 0.20]

This represents:

Stories about space

PART - 9 : L2 Distance

```
SELECT
  book_id,
  title,
  author,
  genre,
  ROUND(
    SQRT(
      POW(
        CAST(
          JSON_EXTRACT(embedding, '$[0]')
          AS DECIMAL(10,6)
        ) - 0.90,
        2
      )
      +
      POW(
        CAST(
          JSON_EXTRACT(embedding, '$[1]')
          AS DECIMAL(10,6)
        ) - 0.80,
        2
      )
      +
      POW(
        CAST(
          JSON_EXTRACT(embedding, '$[2]')
          AS DECIMAL(10,6)
        ) - 0.20,
        2
      )
    )
  )
```

```

),
4
) AS l2_distance
FROM Books
WHERE embedding IS NOT NULL
ORDER BY l2_distance ASC
LIMIT 3;
OUTPUT:

```

The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 204 to 219. The query is as follows:

```

204      2
205      )
206
207      ),
208
209      4
210
211      ) AS l2_distance
212
213      FROM Books
214
215      WHERE embedding IS NOT NULL
216
217      ORDER BY l2_distance ASC
218
219      LIMIT 3;

```

The bottom pane shows the 'Result Grid' with the following data:

	book_id	title	author	genre	l2_distance
▶	1	The Martian	Andy Weir	Science Fiction	0
	8	Enders Game	Orson Scott Card	Science Fiction	0.0735
	2	Dune	Frank Herbert	Science Fiction	0.0866

The left sidebar shows a 'Navigator' with a 'SCHEMAS' section containing a search bar and a list of databases: 2520080069_ananya, bankdb, kludb, and sys. The bottom left shows 'Administration' and 'Schemas' tabs, with 'Schemas' selected. The bottom right shows 'Result Grid', 'Form Editor', and 'Field Types' tabs, with 'Result Grid' selected. The bottom status bar shows 'Result 5 x' and 'Read Only'.

PART-10: Cosine similarity

```
SELECT
  book_id,
  title,
  author,
  genre,
  ROUND(
    (
      CAST(
        JSON_EXTRACT(embedding, '$[0]')
        AS DECIMAL(10,6)
      ) * 0.90
      +
      CAST(
        JSON_EXTRACT(embedding, '$[1]')
        AS DECIMAL(10,6)
      ) * 0.80
      +
      CAST(
        JSON_EXTRACT(embedding, '$[2]')
        AS DECIMAL(10,6)
      ) * 0.20
    )
    /
    (
      SQRT(
        POW(
          CAST(
            JSON_EXTRACT(embedding, '$[0]')
            AS DECIMAL(10,6)
          ),
          2
        )
        +
        POW(
          CAST(
            JSON_EXTRACT(embedding, '$[1]')
            AS DECIMAL(10,6)
          ),
          2
        )
      )
    )
  )
```

```

    )
    +
    POW(
        CAST(
            JSON_EXTRACT(embedding, '$[2]')
            AS DECIMAL(10,6)
        ),
        2
    )
)
)
*
SQRT(
    POW(0.90, 2)
    +
    POW(0.80, 2)
    +
    POW(0.20, 2)
)
),
4
) AS cosine_similarity
FROM Books
WHERE embedding IS NOT NULL
ORDER BY cosine_similarity DESC
LIMIT 3;

```

OUTPUT:

Navigator :.....

Query 1 Skill3_Views_triggers_procedur... Skill3_ACID_isolation SQL File 5* x

Limit to 1000 rows

SCHEMAS

Filter objects

- 2520080069_ananya
- bankdb
- kludb
- sys

305
306)
307
308),
309
310 4
311
312) AS cosine_similarity
313
314 FROM Books
315
316 WHERE embedding IS NOT NULL
317
318 ORDER BY cosine_similarity DESC
319
320 LIMIT 3;

Administration Schemas

Information :.....

No object selected

Result Grid

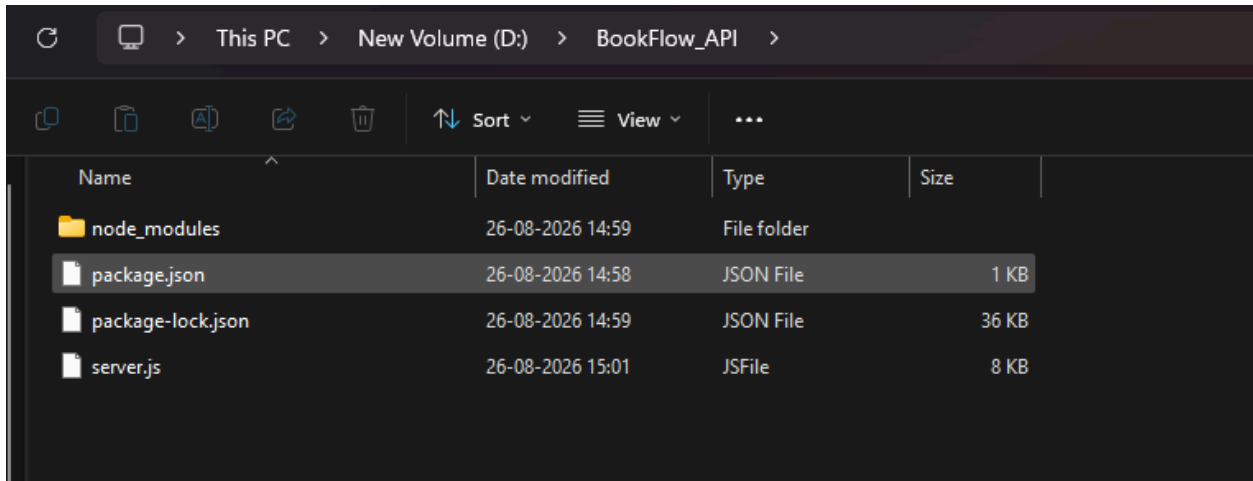
	book_id	title	author	genre	cosine_similarity
▶	1	The Martian	Andy Weir	Science Fiction	1
	2	Dune	Frank Herbert	Science Fiction	0.9986
	8	Enders Game	Orson Scott Card	Science Fiction	0.9983

Result 6 Result 7 x

Read Only

Output :.....

PART 11 : Create API backend



PART 12 : Install Packages

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

D:\BookFlow_API>dir
Volume in drive D is New Volume
Volume Serial Number is C086-3DC2

Directory of D:\BookFlow_API

26-08-2026  14:58    <DIR>          .
26-08-2026  14:58                208 package.json
26-08-2026  14:58                 0 server.js
               2 File(s)                208 bytes
               1 Dir(s)          31,813,632 bytes free

D:\BookFlow_API>npm install

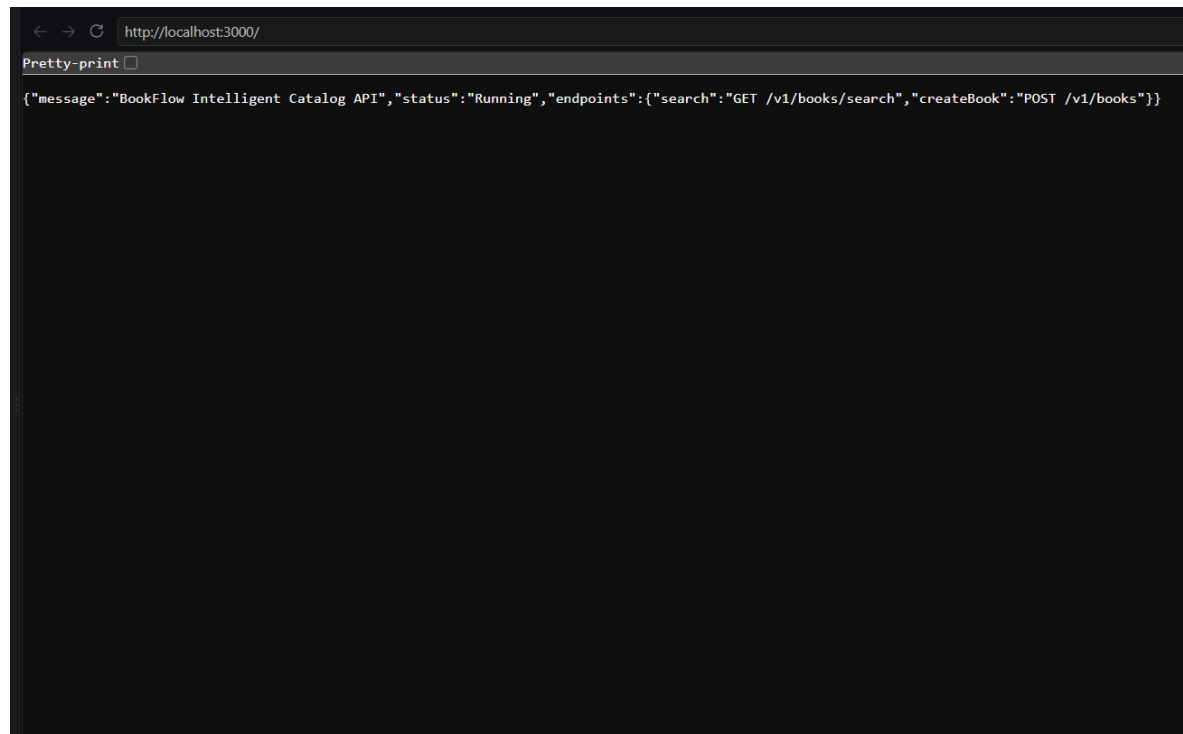
added 78 packages, and audited 79 packages in 6s

29 packages are looking for funding
  run `npm fund` for details

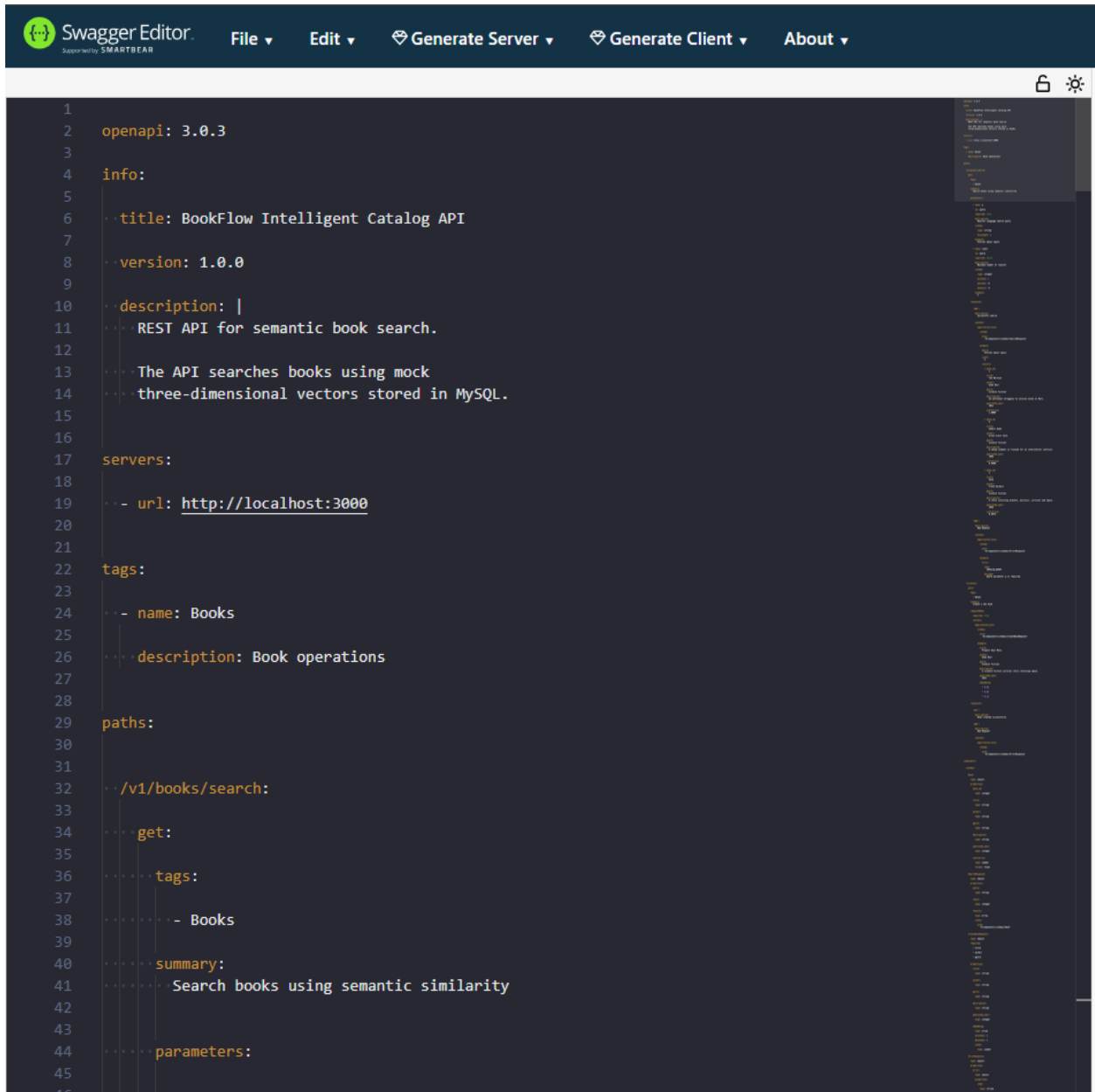
found 0 vulnerabilities

D:\BookFlow_API>
```


PART-13 : Test API in Browser



PART-14: Swagger editor



The image shows the Swagger Editor interface with a dark theme. The top navigation bar includes the Swagger Editor logo, 'Supporting SMARTBEAR', and menu items: File, Edit, Generate Server, Generate Client, and About. The main editor area displays an OpenAPI 3.0.3 specification for 'BookFlow Intelligent Catalog API'. The specification includes an info section with title, version, and description; a servers section with a URL; a tags section with a 'Books' tag; and a paths section with a GET endpoint for '/v1/books/search'. A right-hand sidebar shows a tree view of the specification's structure.

```
1
2 openapi: 3.0.3
3
4 info:
5   title: BookFlow Intelligent Catalog API
6   version: 1.0.0
7   description: |
8     REST API for semantic book search.
9     The API searches books using mock
10    three-dimensional vectors stored in MySQL.
11
12 servers:
13   - url: http://localhost:3000
14
15 tags:
16   - name: Books
17     description: Book operations
18
19 paths:
20   /v1/books/search:
21     get:
22       tags:
23         - Books
24       summary: Search books using semantic similarity
25       parameters:
```

PART - 15 : Swagger editor output

BookFlow Intelligent Catalog API 1.0.0 OAS 3.0

REST API for semantic book search.

The API searches books using mock three-dimensional vectors stored in MySQL.

Servers

`http://localhost:3000` ▾

Books Book operations

GET `/v1/books/search` Search books using semantic similarity ▾

POST `/v1/books` Create a new book ▾

Schemas

Book >

SearchResponse >

CreateBookRequest >

ErrorResponse >

PART - 16 : Swagger test GET

Books Book operations ^

GET /v1/books/search Search books using semantic similarity ^

Parameters Cancel

Name	Description
q * required string (query)	Natural language search query Stories about space
limit integer (query)	Maximum number of results 3

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \  
  'http://localhost:3000/v1/books/search?q=Stories%20about%20space&limit=3' \  
  -H 'accept: application/json'
```

Request URL

```
http://localhost:3000/v1/books/search?q=Stories%20about%20space&limit=3
```

Server response

Code	Description	Links
200	Successful search Media type application/json ▾ Controls Accept header. Example Value Schema <pre>{ "query": "Stories about space", "count": 3, "results": [{ "book_id": 1, "title": "The Martian", "author": "Andy Weir", "genre": "Science Fiction", "description": "An astronaut struggles to survive alone on Mars.", "published_year": 2011, "similarity": 1 }, { "book_id": 8, "title": "Enders Game", "author": "Orson Scott Card", "genre": "Science Fiction", "description": "A young student is trained for an interstellar conflict.", "published_year": 1985, "similarity": 0.998 }, { "book_id": 2, "title": "Dune", "author": "Frank Herbert", "genre": "Science Fiction", "description": "A story involving planets, politics, survival and space." }] }</pre>	No links

PART- 17 : Swagger test 400 error

400	Bad Request Media type application/json ▾ Example Value Schema <pre>{ "error": { "code": "INVALID_QUERY", "message": "Query parameter q is required." } }</pre>	No links
-----	--	----------

PART - 18 : Swagger test POST

POST /v1/books Create a new book ^

Parameters Cancel

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "title": "Project Hail Mary",
  "author": "Andy Weir",
  "genre": "Science Fiction",
  "description": "A science fiction survival story involving space.",
  "published_year": 2021,
  "embedding": [
    0.93,
    0.82,
    0.12
  ]
}
```

Execute Clear

Curl

```
curl -X 'POST' \
  'http://localhost:3000/v1/books' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Project Hail Mary",
    "author": "Andy Weir",
    "genre": "Science Fiction",
    "description": "A science fiction survival story involving space.",
    "published_year": 2021,
    "embedding": [
      0.93,
      0.82,
      0.12
    ]
  },'
```

Request URL

http://localhost:3000/v1/books

Server response

Code	Details
Undocumented	Failed to fetch. Possible Reasons: - CORS - Network Failure - URL scheme must be "http" or "https" for CORS request.

Responses

Code	Description	Links
201	Book created successfully	No links
400	Bad Request	No links

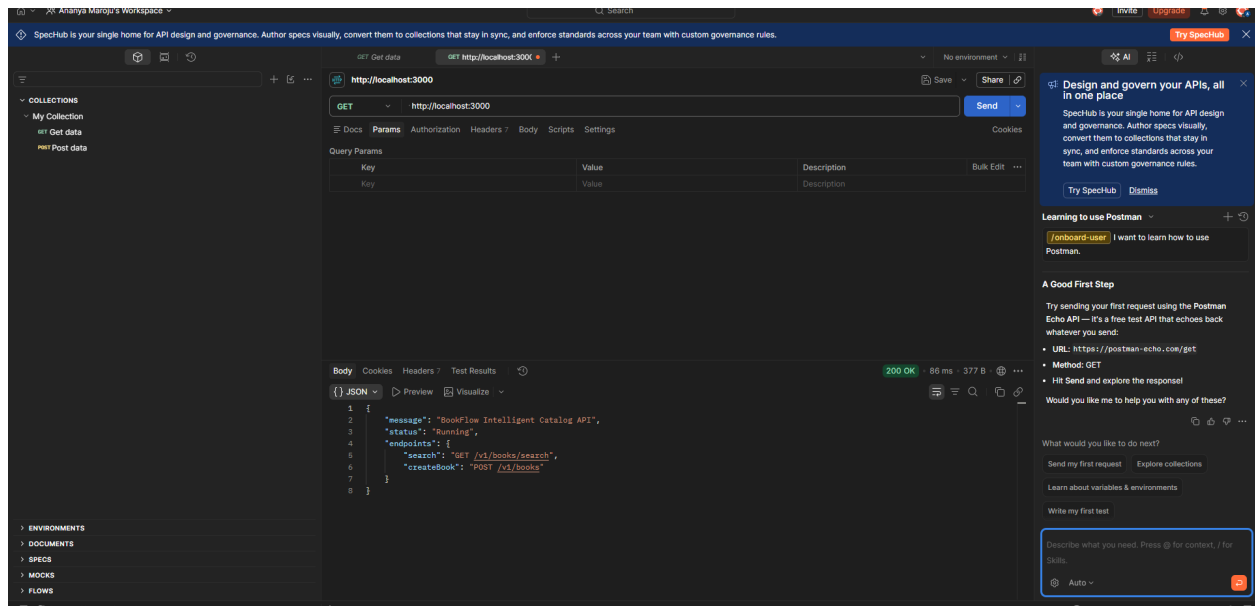
Media type application/json

Example Value | Schema

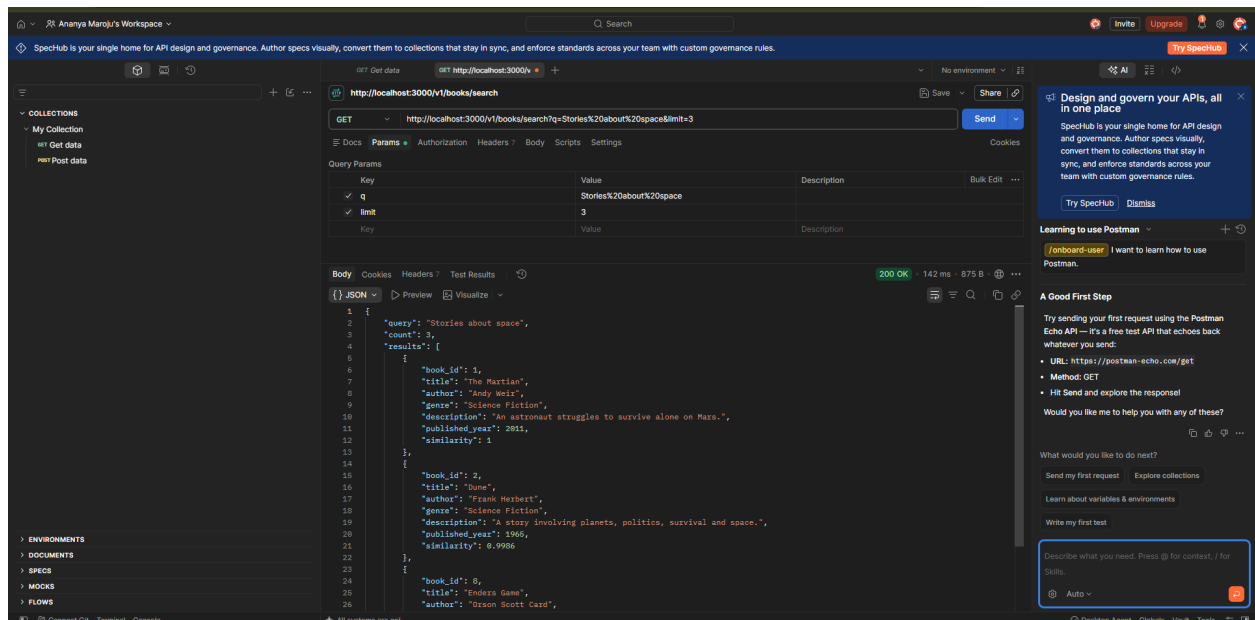
```
{
  "error": {
    "code": "string",
    "message": "string"
  }
}
```

PART- 19 : Postman testing

TEST 1 - API status



TEST 2 - Semantic search



TEST 3 - Missing q

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/v1/books/search`. The query parameters table shows a single parameter `limit` with a value of `3`. The response is a 400 Bad Request with the following JSON body:

```
1 {
2   "error": {
3     "code": "INVALID_QUERY",
4     "message": "Query parameter q is required."
5   }
6 }
```

The right sidebar contains a "Design and govern your APIs" panel, a "Learning to use Postman" panel, and a "A Good First Step" panel.

TEST 4 - Invalid limit

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/v1/books/search?q=space&limit=100`. The query parameters table shows two parameters: `q` with a value of `space` and `limit` with a value of `100`. The response is a 400 Bad Request with the following JSON body:

```
1 {
2   "error": {
3     "code": "INVALID_LIMIT",
4     "message": "limit must be between 1 and 50."
5   }
6 }
```

The right sidebar contains the same panels as the previous screenshot.

TEST 5 - Create Book

The screenshot shows the Swagger UI interface for a workspace named "Ananya Maroj's Workspace". The main area displays a POST request to the endpoint `http://localhost:3000/v1/books`. The request body is a JSON object with the following fields: `title` ("Project Hail Mary"), `author` ("Andy Weir"), `genre` ("Science Fiction"), `description` ("A science fiction survival story involving space."), `published_year` (2021), and `embedding` ([0.93, 0.82, 0.12]). The response body is a JSON object with `message` ("Book created successfully") and `book` (an object containing the same fields as the request body). The status bar indicates a "201 Created" response with a 101 ms duration and 494 B of data.

```
POST http://localhost:3000/v1/books

{
  "title": "Project Hail Mary",
  "author": "Andy Weir",
  "genre": "Science Fiction",
  "description": "A science fiction survival story involving space.",
  "published_year": 2021,
  "embedding": [0.93, 0.82, 0.12]
}
```

```
{
  "message": "Book created successfully",
  "book": {
    "book_id": 11,
    "title": "Project Hail Mary",
    "author": "Andy Weir",
    "genre": "Science Fiction",
    "description": "A science fiction survival story involving space.",
    "published_year": 2021,
    "embedding": [
      0.93,
      0.82,
      0.12
    ]
  }
}
```

TEST 6 - Invalid POST

The screenshot shows the Swagger UI interface for a workspace named "Ananya Maroj's Workspace". The main area displays a POST request to the endpoint `http://localhost:3000/v1/books`. The request body is a JSON object with only the `title` field ("Test Book"). The response body is a JSON object with an `error` field containing `code` ("INVALID_BOOK") and `message` ("title, author and genre are required."). The status bar indicates a "400 Bad Request" response with a 9 ms duration and 327 B of data.

```
POST http://localhost:3000/v1/books

{
  "title": "Test Book"
}
```

```
{
  "error": {
    "code": "INVALID_BOOK",
    "message": "title, author and genre are required."
  }
}
```

PART-20 : Verify data after POST

SELECT

book_id,
title,
author,
genre,
published_year,
embedding

FROM Books

ORDER BY book_id;

OUTPUT:

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
315 WHERE embedding IS NOT NULL
316
317
318 ORDER BY cosine_similarity DESC
319
320 LIMIT 3;
321
322 • SELECT
323     book_id,
324     title,
325     author,
326     genre,
327     published_year,
328     embedding
329 FROM Books
330 ORDER BY book_id;
```

The result grid displays the following data:

book_id	title	author	genre	published_year	embedding
3	The Hitchhikers Guide to the Galaxy	Douglas Adams	Science Fiction	1979	[0.80, 0.70, 0.30]
4	Pride and Prejudice	Jane Austen	Romance	1813	[0.10, 0.20, 0.90]
5	The Great Gatsby	F. Scott Fitzgerald	Classic	1925	[0.15, 0.30, 0.80]
6	Foundation	Isaac Asimov	Science Fiction	1951	[0.95, 0.65, 0.20]
7	The Hobbit	J.R.R. Tolkien	Fantasy	1937	[0.30, 0.75, 0.90]
8	Enders Game	Orson Scott Card	Science Fiction	1985	[0.88, 0.85, 0.15]
9	Harry Potter and the Sorcerers Stone	J.K. Rowling	Fantasy	1997	[0.20, 0.55, 0.95]
10	Twenty Thousand Leagues Under the Seas	Jules Verne	Adventure	1870	[0.70, 0.90, 0.40]
11	Project Hail Mary	Andy Weir	Science Fiction	2021	[0.93, 0.82, 0.12]
•	NULL	NULL	NULL	NULL	NULL

The interface also shows a left sidebar with a 'SCHEMAS' panel and a bottom status bar indicating 'Books 8'.